

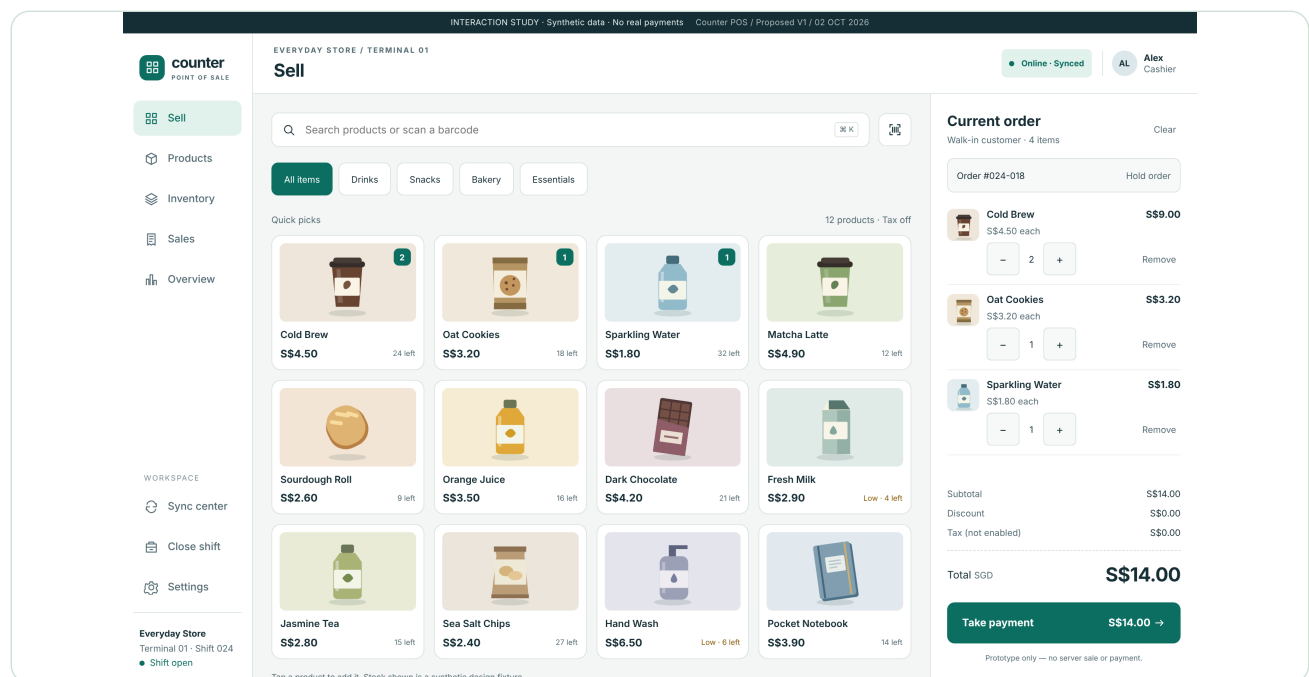
A small counter. A complete system.

PWA Point of Sale

Product & Engineering Handbook

Prepared for Yap Wei Jun · 02 October 2026

English implementation specification + 中文导读



Small-retail workflows. Explicit transaction boundaries.

Product requirements, money and stock rules, offline recovery, API/schema contracts, testing, delivery and operational guidance.

PROPOSED BASELINE · NOT A PRODUCTION APPLICATION

Working identity only. Synthetic examples. Manual external-payment recording is not provider verification.

READING MAP

From product to proof.

Read the business rules before implementing the checkout screen. Screens and the interaction prototype are a visual study, not evidence of a running backend.

中文导读：这个 POS 要怎样做	3
01 · Product requirements	5
02 · Business rules and transaction invariants	7
03 · Screens, interaction and responsive behavior	10
04 · Visual design system	13
05 · Architecture and module boundaries	15
06 · Offline capability and synchronization protocol	17
07 · Database model and persistence contract	20
08 · API and integration boundary	23
09 · Security, privacy and recovery	25
10 · Test strategy and acceptance gates	27
11 · Implementation roadmap and work breakdown	29
12 · Deployment, environments and operating runbook	31
13 · Future ERP integration without ERP dependence	33
14 · GitHub portfolio and demonstration	34
15 · Decision log, assumptions and risk register	36
16 · Source register and provenance	38
17 · Asset guide and handoff conventions	40

Canonical files: editable Markdown in docs/; machine-readable contracts in specs/. The companion UI Design Atlas contains the complete wireframe and interface set.

中文导读：这个 POS 要怎样做

1. 项目定位

你的目标不是再做一个 ERP，也不是复制整个 Odoo，而是把你熟悉的商品、销售和库存流程，做成小店店员容易操作的收银产品。暂名 **Counter POS**；这个名字、技术栈和 repo 名称都是本次建议，尚未成为你确认的项目决策。

第一版以一间店、一个库存地点、一个正在收银的设备为基准。后台可以另开浏览器管理商品，但不能假设多部断网设备同时卖最后一件商品仍然不会超卖。几样或几十样商品即可展示完整流程；测试数据另外覆盖 500 个商品，用来验证性能，不是宣称已经达到的性能成绩。

2. 第一版要交付什么

完整业务链是：建立商品 → 登记进货数量 → 登录并开班 → 点商品加入购物车 → 收款并保存销售 → 开收据 → 产生库存 movement → 查看销售 → 交班对账。

店员只需要看到商品、购物车、付款和清楚的状态。商品成本、调库存、退款批准和设置留在后台。桌面和平板采用商品区加右侧订单区；手机采用商品区加底部“查看订单”，再进入全屏订单和付款。不要把 ERP 那种很多字段的表单直接搬进收银界面。

3. Product 不等于 Inventory

Product / 商品资料回答“这是什么商品、卖多少钱”。**Inventory / 库存**回答“现在还有多少”。

商品表不应该成为随意更改库存的入口。收到 20 件写入 +20 的库存流水；卖出 2 件写入 -2；退回且可以再卖的 1 件写入 +1。库存余额是流水的可重建汇总，更新销售、付款和库存必须放在同一个服务器 transaction 中。历史售价和税额保存于销售明细，不受今天修改商品价格影响。

4. 离线不是无限制

离线功能是在之前已联网登录、下载商品、开班和获准离线操作之后，继续登记现金销售。第一次登录、修改价格、进货、库存调整、退款及正式关班要求联网。

离线销售会显示 **Saved on this device / 已存本机** 和待同步数量，不能假装已经同步到服务器。重开页面、恢复网络、回到应用和手动点击同步都要触发尝试。Background Sync 仅作额外优化，不作为所有浏览器都具备的前提。浏览器储存空间也可能不足或被用户清除，所以不能承诺本机资料永远不会丢失。[S03, S04]

离线付款只开放 Cash。Card 和 PayNow 在 V1 是店员已经在外部终端或商户渠道确认后手动登记的方式，不是系统自动验证到账。不能看到顾客截图就当作可靠的收款结果；真实支付集成应另立项目范围。

5. 防止重复收款和重复扣库存

点两次“完成销售”、网络 timeout 或同步重试，不应该生成两笔销售。每一笔本机销售有固定 UUID，后端保存这个 ID 和内容摘要。同样 ID、同样内容只返回原结果；同样 ID、不同内容必须报冲突。

Idempotency / 幂等：同一个请求重试多次，业务只记一次。它不是不需要测试的保证。测试必须真的模拟服务器保存成功但响应丢失，然后再重试，核对销售、付款和库存流水都只有一份。

6. 退款、作废和交班

购物车还没收款，可以取消草稿。已完成销售不可直接删除，应通过关联原单的退款记录反向处理。退款数量不能超过原购买数量，已退过的不能再退。是否回库存由商品实际可否再售决定。

交班不是只显示今日销售总额。预计现金 = 开班备用金 + 现金销售净额 - 本班现金退款 + 现金存入 - 现金取出。现金找零不是销售收入。待同步销售未处理完时，可以保存点钞草稿，但不能假装完成最终对账。

7. 这套 ZIP 怎样使用

先看 `START-HERE.html` 和 `UI-Design-Atlas.pdf`。你可以用 `prototype/index.html` 体验商品加入订单和模拟现金结账；它只用于讨论 interaction，刷新会清空，不是正式系统。

把整个目录交给 AI Agent，并让它先读 `prompts/IMPLEMENTATION-TASK.md`。Agent 必须先 audit 环境、建立隔离 branch/worktree、确认依赖和测试，然后逐阶段实现；不能看到 SQL 文件就直接对你的 ERP 数据库执行，也不能自行部署到现有生产服务。

8. 简历怎样展示

完成后最有价值的演示是：一笔正常现金销售、一笔断网后重连的销售、一次重复提交测试、一次退款及库存对照。README 展示真实运行证据、架构取舍和已知限制。现在只能说“设计规格与交互原型完成”，不能写“已经上线使用”或虚构测试通过率。

术语：**wireframe** = 线框图；**design tokens** = 统一颜色、字号与间距变量；**outbox** = 待同步队列；**stock movement** = 库存流水；**reconciliation** = 对账；**acceptance criteria** = 验收条件。

01 · Product requirements

中文重点：先完成小店的一天，而不是把所有 ERP 模块搬进来。

Intent and evidence

The user requested a standalone PWA POS portfolio project on GitHub, intended for small businesses with a small catalogue and a good design. This kit is a proposed response to that brief. Odoo documentation is a workflow reference, not evidence that its source code or offline protocol was audited. Its browser-based POS positioning and daily operational workflow inform the broad product categories only. [S01]

Personas and jobs

Persona	Job	Design consequence
Cashier	Sell a few items quickly and recover from mistakes	Large product targets; persistent total; minimal checkout fields
Owner / manager	Maintain stock, verify sales and close a shift	Separate back office; accountable adjustments; transparent sync exceptions
Portfolio reviewer	Understand the system in minutes	One-click synthetic demo, visible architecture and reproducible tests

Scope baseline

Single business per deployment; one store and stock location; one active selling device and one writable POS tab. Multiple owner/cashier users may exist, but there is one open cash shift at a time. Stock is tracked in whole units only. The initial dataset contains 12 synthetic convenience-store products; benchmark fixtures expand to 500 products. This is a planning boundary, not a tested maximum.

One currency is selected at setup (SGD in examples; MYR is another proposed configuration). Freeze currency after the first posted transaction. Tax is off in sample data. A configurable exclusive tax calculation may be enabled only after the owner verifies applicable treatment; statutory fiscalization and e-invoicing are outside V1.

Functional requirements

ID	Requirement	Release
FR-01	Online sign-in, roles, registered terminal, open cash shift	V1
FR-02	Create/edit/archive products, unique SKU/barcode, categories, price history	V1
FR-03	Receive stock and post reasoned adjustments to an immutable ledger	V1
FR-04	Product search, category chips, keyboard-wedge barcode input	V1

ID	Requirement	Release
FR-05	Cart quantity, remove/undo, locally held carts, line discount with permission	V1
FR-06	Cash tender/change; one manually recorded external method per online sale	V1
FR-07	Receipt, sale history and manager-authorized online partial/full refunds	V1
FR-08	Local atomic sale persistence, durable outbox, idempotent posting and visible recovery	V1
FR-09	Daily gross/net sales, refunds, orders, average ticket, low stock	V1
FR-10	Shift cash count, variance reason, final close only after reconciliation	V1
FR-11	Installable shell, user-controlled updates, responsive accessible interaction	V1
FR-12	Backup/restore runbook, audit trail, CSV export and isolated demo	V1
FR-13	ERP adapter contracts without dependence on an ERP	Later

Do not implement split tender, customer debt, loyalty, restaurant tables, kitchen printing, weighed products, product variants, multi-store inventory, purchase orders, accounting, ecommerce, automatic card processing, PayNow verification or AI agents inside checkout in V1. Keep their extension points without showing dead buttons in production.

User journeys

Owner setup: create store → select currency/timezone → verify tax-off default → create categories/products → receive opening stock → create cashier → enroll device.

Cashier day: sign in online → verify catalogue/device → open shift with float → search/scan/tap → review cart → enter tender → confirm physical cash → durable save → show receipt → continue selling → count cash → synchronize → close.

Failure recovery: sell cash offline under valid permit → show local receipt and pending badge → reconnect/re-authenticate as necessary → resend identical document → obtain canonical receipt → reconcile local pending adjustments → resolve exceptions without deleting evidence.

Success criteria: targets, not measurements

The eventual implementation should let a new tester complete a three-product cash sale without training, recover an acknowledged local transaction after a normal reload, and retry a server-committed sale without duplication. Proposed performance budgets: local add-to-cart p95 below 100 ms for 500 cached products; search response below 150 ms; warm shell usable within 2 seconds on the declared test device. Measure with browser/version/hardware, dataset, network profile and run count in the report; do not publish an unsupported badge.

Definition of ready for development

Record the target environment and repository, accepted scope exceptions, dependency lockfile policy, screenshot sizes, risk owner, payment boundary and release gates. The temporary project name and styling can change independently of domain behavior.

02 · Business rules and transaction invariants

中文重点：钱、库存、退款和同步必须一致。以下是本项目规则，不是税务或支付合规保证。

Money and quantity

All monetary values are integers in the configured currency's minor unit. V1 admits two-decimal currencies only. JSON values are bounded safe integers: each nonnegative amount is at most 1,000,000,000 minor units; reject a document whose intermediate or final totals exceed this cap. Compute with integer/BigInt intermediates, never binary floating-point prices. Whole quantities are integers 1–999 per line; at most 100 lines per sale. Monetary aggregates and SQL storage use BIGINT; serializers validate the API range.

Use a stable line_id so the same product can appear at distinct approved prices. A duplicate tap normally increments the existing equivalent-price line. SKU, description, unit price, tax basis points and price revision are snapshotted on the sale line.

For each line: gross = quantity × unit_price_minor; net = gross – discount_minor. For exclusive tax, tax_minor = floor((net × tax_bps + 5000) / 10000), using integer half-up rounding for nonnegative net. Line total = net + tax. Sale gross, discount and tax are sums of line values; amount due = gross – discount + tax. No document-level discount, tax-inclusive prices, compound taxes, cash rounding or tips in V1.

Worked synthetic fixture: 2 cold brew × 450 + 1 oat cookies × 320 + 1 sparkling water × 180 = 1,400 minor units. Discount 0; tax disabled; due SGD 14.00. Cash tender 2,000; change 600; payment amount applied 1,400. Never treat the 2,000 tender as sales revenue. A separate artificial 500-bps example for calculation tests is not a regional tax recommendation.

Sale and payment state

Cart states: DRAFT → PAYMENT_REVIEW → LOCAL_COMMITTED → SERVER_POSTED. Cancel is allowed only before LOCAL_COMMITTED. Sync state (pending/sending/acked/retry/needs_review) is separate from commercial state. A saved sale cannot return to a mutable cart. Local and server receipt identifiers may both exist; do not replace the underlying UUID.

V1 accepts exactly one payment row per sale. CASH stores applied amount, tender and change.

CARD_MANUAL and PAYNOW_MANUAL store applied amount, operator verification time and a non-sensitive reference; tender equals applied amount, change is zero. They require the online workflow. The visible label is “Recorded externally”, not “Provider verified”. A QR graphic or customer screenshot alone is not verification. No PAN, CVV, card token or bank credentials are collected.

Price changes and offline authorization

Cart pricing uses the last accepted catalogue version. Going to payment refreshes price availability when online; a changed price requires explicit review before receiving money. Once physical payment is taken and the local document commits, a later price update must not silently rewrite that sale.

The server validates online quoted prices or signed offline-permit limits against retained price revisions. Legitimate completed offline cash sales with older permitted prices are posted with their original amounts. Invalid signatures, unknown revisions or excessive discounts enter NEEDS_REVIEW; preserve the original payload and cashier receipt. Cash may already have changed hands, so never auto-delete or auto-charge again.

Inventory invariant

A stock movement is the authoritative event; stock_balances is a rebuildable projection. Both movement insertion and balance change occur in the same database transaction as a posted sale or refund. Opening stock is a RECEIPT movement, not an unexplained product quantity. Types: RECEIPT (+), SALE (-), RETURN (+ if restockable), ADJUSTMENT (signed nonzero).

Draft and held carts do not reserve stock. The POS warns using the latest server balance plus unacknowledged local deltas. Do not apply the same local delta again after its sale is acknowledged. A new online sale is blocked before payment if insufficient stock is known. An authentic already-completed offline sale may create a negative balance on replay and an owner exception; rejecting reality would hide cash and stock movement. The owner resolves the discrepancy through a reasoned movement, not editing historical sales.

Adjustments require current balance_version, counted quantity or explicit delta, reason and actor. Conflicting counts return 409 and require a refresh/recount. Do not use a stale count to overwrite newer sales.

Refunds

Refunds are manager-only, online, linked to a SERVER_POSTED original sale. Lock the original lines and cumulative returned quantities in a stable order. Refund quantity must not exceed remaining quantity; refund currency and tax treatment follow original snapshots. For rounding, refund the difference between cumulative entitled amounts before/after this refund; the final refund receives the remaining cents so total refunds equal—but never exceed—the original total.

Each refund line states restock=true/false. Resalable returns add stock; damaged returns do not. Refunding cash affects the currently open shift, not the original shift. Manual electronic refunds record an externally executed refund only after verification; there is no automatic money movement. An entire refund, its payment record, restock movements and audit event commit atomically. Use a stable refund UUID for retries. Posted sales/refunds cannot be deleted; cancellation before payment is not a refund.

Shift and reporting

One open shift per store and terminal. Opening requires online authentication and a starting cash float. Expected drawer = opening float + CASH applied sale amounts – CASH refund amounts + cash-in – cash-out. Variance = actual cash count – expected drawer. Every nonzero variance needs a reason. The included dashboard sample shows gross 486.00, refunds 18.00, net 468.00 over 36 completed sales; average ticket = gross/36 = 13.50. Unsynced amounts are shown separately, never mixed into server-confirmed totals without a label.

Use the shift's server-assigned business_date in the configured timezone for sale reporting; client clock is evidence, not the authority. Recognize refunds on the current refund shift's business date. Store UTC timestamps plus business_date. Final shift close is online, requires no pending/review documents and a terminal reconciliation acknowledgement. Local cash-count drafts do not close the server shift.

Audit and identity

Server derives store/role from authentication; never trusts a body shop_id. Enforce uniqueness for SKU, normalized barcode, client sale UUID and canonical receipt number within store. Use UUIDs for identity and a server sequence for receipt display; sequence gaps are acceptable and documented. Retain original actor/device, reason, UTC time, request ID and linked entities for every material action.

03 · Screens, interaction and responsive behavior

中文重点：收银页面以最快完成销售为主；后台才展示管理字段。

Navigation and layout

Primary destinations: Sell, Products, Inventory, Sales, Overview. Shift and Sync are persistent utilities; Settings is manager-only. The desktop side rail includes labels, not unexplained icons. At 1024px and above, keep a 360–400px order panel and a flexible product grid. At 768–1023px, use a compact labeled rail and 320px cart when space permits. Below 768px, show a two-column product grid and a fixed “View order · total” action; cart and payment become full-screen sheets.

The POS header stays visible: an explicit exception to the house standard's hide-on-scroll default because scan/search, connectivity and cashier context are transaction controls. Product lists and long back-office pages may use a hide/reveal header if it does not hide critical status. Respect device safe areas and browser zoom. Never set user-scalable=no.

Screen inventory

ID	Screen	Content and actions	Important states
UI-01	Sign-in / open shift	Account sign-in, device identity, opening float	Expired session, offline first launch, occupied terminal
UI-02	Sell	Search/scan, category chips, product grid, cart, fixed total	Ready, loading, no results, stale catalogue, sold out
UI-03	Mobile order	Full line descriptions, quantity controls, remove/undo, total	Empty, stock warning, insufficient permission
UI-04	Payment	Due, method, numeric tender, change, explicit confirm	Underpaid, duplicate click disabled, local save failed
UI-05	Receipt	Snapshot lines, local/canonical ID, applied/tender/change, print	Saved locally, synced, print failed, reprint
UI-06	Products	Search, category/status filters, SKU/price/stock, editor drawer	Validation conflict, archive instead of delete
UI-07	Inventory	On-hand, ledger, receive/adjust panel, version and reason	Stale count, negative-stock exception
UI-08	Sales / return	Receipt search, date range, method, detail, manager refund	Local-only sale, already refunded, reference unavailable
UI-09	Overview	Gross/refund/net/orders, top items, low stock, sync disclosure	No sales, delayed data, mixed business dates

ID	Screen	Content and actions	Important states
UI-10	Sync center	Pending/acked/retry/review, last attempt, recovery actions	Authentication needed, conflict, server unavailable
UI-11	Close shift	Expected cash, actual count, difference, reason	Pending sync blocker, count draft, manager review
UI-12	Settings / updates	Store, currency/tax, device, backup help, build update	Update waiting, checkout busy, migration blocked

Product selection and cart

Search by normalized product name, SKU and exact barcode. A keyboard-wedge scanner is the baseline input device; camera scanning is a later capability requiring camera permission and device tests. Preserve keyboard focus; do not capture scanner-like typing while a text or money input is active. Unknown barcodes show “Product not found” with search; do not silently create products.

A product card displays its image or initial, name, price and stock hint. Images are secondary to names. Tap increments quantity and briefly confirms the addition without opening a dialog. Quantity actions have a 44px target; disable minus at the minimum or turn it into explicit remove. Remove offers a short Undo action before checkout. A price override is not an ordinary cashier action.

Held carts remain on the current device and make no stock reservation. Restore refreshes pricing before payment. “Clear order” confirms if nonempty. There is no customer form in the primary flow; anonymous walk-in is the default.

Payment and receipt interaction

Payment is a focused task, not another full admin page. Show amount due once at large size, available methods with explanatory text, tender input and change. Underpayment disables confirmation. Enter submits only when the focused form is valid and the user has reviewed the amount; shortcuts must not accidentally finalize a sale while scanning.

After confirmation, disable repeat submission, persist the local document atomically and only then display success. “Saved on this device — waiting to sync” uses an amber status and does not imply server acknowledgement. An online acknowledged sale displays the canonical receipt and “Synced”. If saving fails, keep the order, tell the cashier that nothing was saved, and stop the handover flow.

Printing opens browser print as the V1 baseline. A reprint must not create a new sale. Printer failure does not roll back an already accepted sale. 80mm receipt CSS is proposed; physical thermal printer fit, cash drawers and silent printing are not certified by a browser preview.

Back office and errors

Products editor: name, SKU, barcode, category, price, cost, low-stock threshold, active. Explain duplicate field errors inline and preserve input. Create product does not receive stock. Inventory adjustment shows current quantity, counted quantity, computed delta and mandatory reason before posting.

A refund form lists remaining returnable quantity and restock choice per original line. Show the final refund amount before external refund/cash handover. A read-only sale detail must still expose useful history to cashiers without refund permission.

Sync errors use plain language plus expandable technical details. Recovery never offers “Delete pending sale”. Provide retry, reauthenticate, export recovery package and contact manager. Product/stock administration stays disabled offline with the reason adjacent to the action.

Accessibility and microcopy

Use semantic buttons, dialogs and headings; label icon-only controls; trap and restore dialog focus; expose important status through aria-live without announcing every total change repeatedly. Respect reduced motion. Design target is WCAG 2.2 AA, not an asserted audit result. The 44px house touch target is stricter than WCAG 2.2's 24px minimum-target rule with exceptions. [S07]

Preferred words: “Take payment”, “Cash received”, “Change”, “Saved on this device”, “Waiting to sync”, “Recorded externally”, “Return items”, “Close shift”. Avoid “Success” for a server write that has not happened. Language switches must not change money, IDs or receipt snapshots.

04 · Visual design system

中文重点：轻量、清楚、触控友好；不要用很多装饰把总额和付款按钮淹没。

Direction

Counter POS is a working identity for this kit only. Use an understated, warm-white workspace, dark ink typography and a teal action color. Products get pale illustrated backgrounds; stock/sync status stays textual and restrained. A compact brand mark resembles a counter with a receipt. Do not copy Odoo assets, payment-provider logos or a retailer's identity.

The proposed palette is ink #152D35, muted text #536A73, surface #FFFFFF, canvas #F3F6F5, border #D8E2DF, primary #0B6E61, primary-dark #07564C, primary-tint #E1F1EC, warning #8A5700 on #FFF2D6 and danger #B42318 on #FDEAE7. Tokens are in `design/tokens.json` and `design/tokens.css`. Contrast must be verified on actual components, including disabled states and focus outlines; palette selection alone is not an accessibility certification.

Typography and spacing

Use a local/system sans-serif stack: Inter when installed, otherwise system UI, Segoe UI and Noto Sans CJK for Mandarin. No font binaries are distributed in this ZIP. Browser rendering varies slightly across OS. Body text 16px; supporting text 13–14px; screen title 26–30px; total 28–36px. Tabular numerals for money and quantity. Do not reduce the entire app to ERP-style 9pt text.

Spacing uses a 4px base: 4/8/12/16/24/32/48. Product cards use 16px padding, 16px corners and a low-contrast border. Buttons use 10–12px corners. Avoid heavy drop shadows; elevation is reserved for overlays and mobile order bars. Shadows must not be the only boundary indicator.

Components and states

Primary button: one dominant action per task, 48–56px high. Secondary: white surface and border. Destructive: text plus confirmation when the action is irreversible. Loading buttons retain width, show progress and reject repeat activation. Text inputs are 44–48px high and at least 16px text on mobile. Money inputs use a decimal keyboard but validate locale-normalized text before converting to minor units.

ProductCard has default, keyboard focus, hover, selected feedback, out-of-stock and unavailable states. CartLine has quantity adjustment, removing/undo, permission-denied and price-changed states. StatusBadge always contains text: Online, Offline, Pending 3, Needs review. Never rely on green/red alone.

Dialogs have a visible title, labeled close action, a predictable footer and focus restoration. On phones the order/payment flow occupies the viewport instead of stacking small nested modals. Toasts avoid covering totals and remain long enough to read; critical errors persist inline.

Responsive tokens

Phone content gutter 16px; tablet 20px; desktop 24–32px. Keep the mobile CTA above `env(safe-area-inset-bottom, 0px)`. Use 100dvh with a compatible fallback, opaque top safe-area surfaces and scrollable middle content. Do not force a fixed viewport height that hides fields behind the software keyboard. Input focus should scroll into view.

At 390×844 and 430×932: two-column product grid, no persistent desktop rail, full-width order sheet. At 768×1024: two-column products with compact or collapsed cart depending on available width. At 1024×768 and 1440×900: product/cart split. Reflow to the phone pattern when zoom reduces the usable CSS width.

Motion and themes

Use 120–180ms opacity/transform transitions for small UI feedback. Honor prefers-reduced-motion. No animation during price calculation or payment confirmation. V1 is light-only; dark theme is explicitly deferred, so no untested “dark-mode supported” badge. Both manifest theme color and shell top surface match light-mode ink/surface choices.

Assets and authority

PNG screens show a synthetic store with tax disabled. Their totals match the canonical demo fixture. SVG wireframes express hierarchy and behavior; they are editable source assets, not a ready-made Figma component library. The AI exploratory poster is non-authoritative: generated names, license claims, percentages, schema labels and payment capability text are not implementation requirements. Follow this specification and tokens instead.

Visual acceptance

Review at target sizes, 200% zoom, keyboard navigation, long Mandarin strings, large totals, missing product images, empty cart, storage error and offline mode. No horizontal page overflow; no text truncation that hides the distinguishing part of a product name; no sticky footer over the final line; no small unlabeled icon targets. The atlas is a design review aid, not proof that a production app passes these checks.

05 · Architecture and module boundaries

中文重点：一个清楚的模块化系统比很多微服务更适合这份 portfolio。

Proposed stack and rationale

Use a TypeScript modular monolith: React + Vite for the client, Fastify for the HTTP API, PostgreSQL for durable server state and Dexie as an IndexedDB convenience layer. Share pure money and validation contracts through a separate domain package. Vite and Fastify have official getting-started/API documentation; pin compatible supported versions at implementation kickoff rather than copying an unverified “latest” version into this plan. [S10, S11, S12]

This stack is a recommendation, not a statement about the user's existing proficiency or a repo already created. A CFML backend could implement the same contracts; the portfolio proposal uses TypeScript to demonstrate another delivery environment. Do not add microservices, Kafka, Redis or an AI dependency unless a measured problem justifies them.

Runtime boundary

The browser renders views, stores a bounded catalogue snapshot and holds locally committed sales awaiting acknowledgement. The service worker caches the shell and versioned static assets; it does not intercept and blindly replay arbitrary POST requests. The foreground sync worker drains the explicit outbox. The API authenticates, validates and posts domain commands. PostgreSQL owns sale/stock/audit truth. An integration outbox later delivers ERP events after local business commits.

Serve the application and `/api` behind one HTTPS origin to simplify secure cookies and avoid cross-origin authentication complexity. The API and database remain private behind the reverse proxy/network. Public demo uses a separate environment and synthetic data. Static GitHub Pages may host docs or a browser-only demo; it is not a Node/PostgreSQL host. [S08]

Code structure: implementation target

```
apps/  
  web/src/{app, features, components, db, sync, pwa, i18n}  
  api/src/{plugins, modules, jobs}  
packages/  
  domain/{money, pricing, sales, returns, inventory}  
  contracts/{schemas, errors, events}  
  fixtures/  
infra/{compose, proxy, migrations, backup}  
docs/{adr, runbooks, qa}  
tests/{unit, integration, e2e, fixtures}
```

Feature modules: auth/device, catalogue, inventory, sales, refunds, shifts, reporting and sync. Views never write SQL or calculate authoritative refunds. API routes delegate to application services. Services use repositories inside an explicit transaction boundary. Share pure calculations but do not expose server-only secrets in frontend bundles.

Server posting transaction

Authenticate operator and terminal → enforce store context → look up stable client_sale_id → compare canonical payload hash → validate price revision/permit and totals → lock shift and stock balance rows in product-ID order → insert immutable sale, lines and one payment → append stock movements and update balances → append audit and optional domain event → commit → return canonical receipt and acknowledgement cursor.

The stable unique key handles concurrent duplicate attempts; use transaction retries for recognized serialization/deadlock errors. Never perform a payment-provider call or wait for a human while holding database locks. PostgreSQL row locks last until transaction end and consistent lock ordering reduces deadlock exposure. [S06]

If the response is lost after commit, an identical request returns the existing result. Idempotency records must remain available as long as old offline documents can be replayed. The permanent sale unique key is the final defense; a short-lived request cache alone is insufficient.

Query and data shaping

Catalogue bootstrap is paginated/versioned and excludes product cost for cashiers. Reporting endpoints aggregate server-posted rows by business date and return pending-local counts separately where the client supplies local status. Use cursor pagination for sales/ledger; no unbounded download to render a single screen. Store images as external object keys or controlled relative assets, not enormous base64 values in product rows.

Failure isolation

A printer, report export or future ERP delivery failure cannot reverse a posted sale. Failed sync retains its payload. Failed local storage blocks checkout. A server outage leaves the cached, authorized cash workflow available but disables admin mutations. Invalid authentication stops network posting; reauthentication resumes the same outbox documents. Do not retry by making a new ID.

Trust boundaries

The browser is untrusted. Prices, stock, role, client time and “payment verified” booleans are validated against server policy. Offline authorization is a limited business risk decision, not server-equivalent security. Browser storage has quota and deletion limits; persistence APIs reduce some risks but do not make it a backup. [S04]

06 · Offline capability and synchronization protocol

中文重点：只有成功写入本机 transaction 后才显示“已保存”；只有服务器确认后才显示“已同步”。

Capability matrix

Operation	Online	Offline after readiness
Launch cached app / browse downloaded products	Yes	Yes
First login, enroll device, open a new shift	Yes	No
Edit/hold local cart	Yes	Yes
Cash sale under valid shift/permit	Yes	Yes
Record Card / PayNow external payment	Yes	No
Product price edit / receive / adjust inventory	Yes	No
Refund, final close, restore backup	Yes	No
View locally saved receipt	Yes	Yes, visibly local
Server-wide analytics	Yes	Last snapshot only, labeled stale
Stage a cash-count draft	Yes	Yes; does not close shift

Readiness and permit

Before going offline, complete login, terminal enrollment, catalogue download, open shift and a local write/read self-test. Download a server-signed offline permit scoped to store, user, terminal, shift, catalogue price revisions, allowed methods and discount ceiling. Proposed validity is at most 12 hours and 200 cash sales per permit; these are conservative planning limits to validate with the owner, not platform requirements.

The permit is not an API authentication bearer. Normal reauthenticated session credentials are still needed to upload. Offline unlock cannot securely replace server authentication. Clock rollback/restart can weaken expiry checks; use last trusted server time plus monotonic elapsed time while running, and block new offline sales after a restart when the permit's remaining validity cannot be established conservatively. Cached drafts remain readable. Revocation cannot take effect on a disconnected terminal immediately; document this residual risk.

Local stores and atomic commit

IndexedDB stores: catalogue, price_revisions, shifts, permits, held_carts, local_sales, local_sale_lines, local_payments, local_stock_deltas, outbox, sync_meta. Persist no passwords, PAN/CVV or ordinary API tokens. IndexedDB supports transactional structured storage; application-level boundaries still need explicit design and tests. [S05]

When the cashier confirms cash received, generate sale and line UUIDs once. In a single IndexedDB readwrite transaction, write the frozen sale/lines/payment, matching pending stock deltas and outbox document; remove the mutable cart. Resolve success only after the transaction completes. If quota, serialization or transaction errors occur, abort and show “Sale not saved”. Never clear the cart first. Normal browser durability is not a guarantee against device loss, storage clearing or all power-loss scenarios. [S04]

Outbox contract

Each item has event_id, client_sale_id, schema_version, canonical_payload_hash, frozen payload, created_at, attempts, next_retry_at, state and last_error. States: PENDING → SENDING → ACKED; transient failures become RETRY; permanent/conflicting or permission issues become NEEDS_REVIEW or AUTH_REQUIRED. SENDING leases expire on crash so the exact payload is retried.

Only one tab owns a renewable writer/sync lease, acquired by an atomic IndexedDB compare-and-set transaction with owner and expiry. A second tab is read-only until the lease can safely be claimed. BroadcastChannel/Web Locks may improve coordination when available; server idempotency remains required even with perfect local coordination.

Push protocol

Send one bounded sale document per POST `/api/v1/sales`, with Idempotency-Key equal to client_sale_id. The API derives store identity, canonicalizes allowed fields and computes its own hash. Same identity + same hash returns the original posted outcome; same identity + different hash returns 409 IDEMPOTENCY_CONFLICT. Never trust the client-supplied hash alone.

On acknowledgement, update receipt mapping and outbox state in one local transaction. Reconcile the stock watermark: replace the cached server balance at the returned cursor and apply only local deltas absent from that cursor/acknowledged set. A plain “subtract after sync” produces double deductions. Retain acknowledged local sales until a verified backup/retention policy allows pruning; do not clear them on every version update.

Batching is optional later. If added, each sale remains atomic with a separate result; a partial batch cannot be reported as all successful.

Pull protocol

GET catalogue snapshot/delta uses an opaque cursor, immutable price revisions and archive tombstones. Apply each received page transactionally. Record the next cursor only after the page commits. Expired cursor requires a full bootstrap that replaces catalogue state but leaves local sales/outbox intact. Snapshot switch happens between transactions, not halfway through payment review.

Retry and connectivity

Retry on foreground launch, window focus/visibility, online event followed by a real health probe, manual Sync and an active-app timer. Use exponential backoff with jitter, capped at 60 seconds, respecting Retry-After. Distinguish connectivity, 401, 403, 409/422 and 5xx. A browser online flag alone does not establish server reachability.

Background Sync is an optional enhancement because its browser availability is limited. Do not promise sync while the app is closed or the OS suspends it. [S03] The customer-visible fallback is “Open Counter POS to finish syncing”.

Conflict policy

Price change after local completion: post valid historical snapshot and flag only when outside permit.

Insufficient current stock: post authentic offline sale with negative-stock exception rather than hiding a completed transaction. Revoked/unknown actor, unknown product revision or modified payload: preserve quarantine evidence and require manager review. Closed shift: stop and reconcile; do not silently move cash into today's shift.

Manual recovery export contains schema version, checksums, terminal identity and pending documents; it contains business data, so restrict to managers, warn about sensitivity and prefer encryption. Restore validates checksums and deduplicates by original UUID; never creates new sale identities.

Service worker updates and migrations

Use versioned static cache names and fresh HTML/worker headers. Show waiting update and explicit Update now. Defer activation during payment, nonempty outbox or an incompatible local schema migration. Show an updating overlay, activate once, wait for controller change and reload. Retain at least one compatible rollback build and test $N \rightarrow N+1$ with pending data. House-standard source: [K01]; platform update behavior: [S02].

Offline release blockers

No launch/readiness test, dropped local sale, duplicate server stock movement, unsupported browser assumption, automatic refresh during payment, unbounded retries, negative stock without an exception, offline electronic payment presented as verified, or silent local-data deletion are release blockers.

07 · Database model and persistence contract

中文重点：SQL 是可评审的 reference schema；真正应用还需要 transaction service、权限和 migration 测试。

Schema authority

`specs/schema.sql` is the proposed PostgreSQL relational baseline. It is not an applied migration against the user's data. The SQL includes types, keys, tenant-safe composite references, indexes and selected immutability guards. Cross-row financial invariants are enforced by the application posting service and integration tests; a CHECK constraint cannot validate sums across arbitrary child rows by itself.

Entities and ownership

Entity	Purpose	Key relationship
stores	Currency/timezone and receipt sequence owner	Deployment-level root
users	Named operator and role	Store-scoped, no seed password
devices	Registered terminal identity and revocation	Store-scoped
categories / products	Catalogue, SKU/barcode, current price revision	Store-scoped
product_prices	Immutable pricing/tax revisions	Product and catalogue version
shifts	Opening cash, business date, closed state and count	User + terminal
offline_permits	Signed authorization metadata and validity	Shift + terminal + user
sales / sale_lines	Posted transaction and historical item snapshot	Shift + unique client UUID
payments	One applied tender per V1 sale	Sale; method/verification boundary
refunds / refund_lines	Separate linked return and historical allocation	Original sale and line
refund_payments	Actual refund method record	Refund + current shift
stock_balances	Transactionally maintained, rebuildable quantity	Product + store
stock_movements	Immutable quantity event with concrete origin	Sale line / refund line / admin event
cash_movements	Signed non-sales cash in/out	Current shift
audit_events	Append-only actor/action/request evidence	Store + subject
integration_outbox	Post-commit delivery state for future adapter	Store + unique event ID

Key constraints

All child references carrying `store_id` use composite foreign keys against `(store_id, id)` so an API bug cannot accidentally attach one store's sale to another store's product. Server auth remains the first boundary. SKU and barcode normalization rules are identical at import, API and database boundaries. Empty barcode becomes NULL. Products referenced by history are archived, not physically deleted.

Sales use UUID primary keys and a separate `(store_id, client_sale_id)` uniqueness rule. Receipt numbers are server allocated and unique per store; UUID is the identity. The receipt sequence is monotonic but not promised gapless. Refund idempotency has the same permanent UUID principle.

Store timestamp fields as `TIMESTAMPZ`, business dates as `DATE` and money as `BIGINT`. Clamp API values before conversion to JS numbers. Whole-unit quantity constraints apply to V1. Balance may become negative only through documented completed-offline-sale replay; the database permits this while the service records the exception.

Snapshot rules

A line contains `product_id` and `price_revision_id` for lineage plus `sku_snapshot`, `name_snapshot`, `unit_price_minor`, `discount_minor`, `tax_bps`, `tax_minor` and `line_total_minor`. Editing today's name or price does not change last month's receipt. Product cost is not part of a cashier bootstrap and is not displayed on receipts.

Preserve the accepted document's canonical hash with each sale. Store raw reviewed/quarantined sync documents in a separately access-controlled recovery store during implementation; do not turn malformed payloads into posted sales.

Posting and refund transactions

For a sale, validate all row sums before commit and use stable row lock ordering. The invariant is `sales.total = sum(lines.total) = payment.amount_applied`; cash tender – change = payment amount. SALE movement quantity equals the negative sold quantity exactly once. For a refund, validate cumulative returned quantity/amount while original lines are locked; RETURN movement exists only for `restock=true`.

Stock movements and the balance projection update in the same transaction. Application roles cannot UPDATE/DELETE posted history. Adjustment corrections are compensating events. For ordinary product updates, use version preconditions to avoid lost updates.

Index plan

Indexes cover normalized active SKU/barcode, product category/active, sale shift/business date, sale creation cursor, original-sale refund lookup, stock product/time, unresolved outbox state and audit subject/time. Explain every additional index with an actual query. Do not index all JSON fields by default or perform a ledger SUM for every product card.

Reconciliation queries

At backup/restore or maintenance, compare `stock_balances` to movement sums grouped by store/product; compare sale totals with line and payment sums; compare refunded quantity and amount with original entitlements; verify no cross-store join and no orphaned price revision. All discrepancies generate review output; never silently overwrite the ledger to match a projection.

Migration and seed policy

Use versioned, reviewed forward migrations plus an explicit rollback/restore strategy. Do not auto-run destructive SQL on startup. Test an empty database and one upgraded from the previous release. Production admin creation is an interactive/secret-managed operation; the fixtures contain synthetic products only. The SQL does not include known passwords, public admin credentials or grant scripts that weaken production access.

Deliberate omissions

No financial GL, purchase order, payable, customer-credit balance, item serial/batch, unit conversion, lot valuation or reservation model in V1. ERP mapping remains an adapter concern. Do not reinterpret a stock receipt as a supplier invoice.

08 · API and integration boundary

中文重点：API 是开发合约，不代表这些 endpoint 已经部署。

Contract files

`specs/openapi.yaml` uses OpenAPI 3.1.0 deliberately as an interoperability baseline, not a claim that it is the newest specification. It describes V1 resources, required fields and common errors. [S13] `specs/example-sale.json` is a synthetic valid cash command. The API and UI use the same bounded integer money rules, but the server recalculates all values.

Authentication and security

The production app and API share an origin. Use a Secure, HttpOnly, appropriately SameSite session cookie, short-lived server sessions, CSRF protection on mutations and role checks at every handler. Example cookie name is `counter_session`. Obtain a CSRF token through the authenticated bootstrap; do not place access/refresh tokens in localStorage. Never put an admin key in Vite public environment variables. Session controls and fixation prevention follow the OWASP guidance referenced in [S09].

All paths are prefixed `/api/v1`. The authenticated session determines store scope; resource IDs from another store return not-found/forbidden without revealing metadata. Parameterized SQL and bounded pagination are mandatory.

Endpoint groups

Group	Operations	Authorization
Session	POST login/logout; GET bootstrap	Public login; authenticated remainder
Catalogue	GET products/categories; POST/PATCH products	Cashier read projection; manager write
Inventory	GET balances/ledger; POST receipts/adjustments	Manager writes
Shifts	POST open; GET current; POST count/close; cash movement	Cashier own shift; manager review
Sales	POST frozen sale; GET list/detail; receipt data	Cashier within allowed scope
Refunds	POST linked refund; GET refund detail	Manager, online only
Sync	GET catalogue changes; GET status/probe	Registered authenticated terminal
Reporting	GET daily summary	Manager

The reference OpenAPI deliberately includes the core implementation surface. Explicitly documented later ERP connectors, payment gateways and advanced reports are not claimed as implemented endpoints.

Idempotency, preconditions and errors

Sale/refund creation requires an Idempotency-Key matching the frozen client document UUID. Posting retries must be byte-equivalent after canonicalization. Do not mix a new request body with a reused key. Product and inventory mutations use `expected_version`; stale updates return 409 `VERSION_CONFLICT`.

Standard error shape is `{code, message, request_id, retryable, details}`. Codes include `VALIDATION_ERROR`, `AUTH_REQUIRED`, `FORBIDDEN`, `STORE_MISMATCH`, `SHIFT_NOT_OPEN`, `STOCK_UNAVAILABLE`, `OFFLINE_PERMIT_INVALID`, `IDEMPOTENCY_CONFLICT`, `VERSION_CONFLICT`, `REFUND_LIMIT`, `NEEDS_REVIEW`, `RATE_LIMITED` and `SERVICE_UNAVAILABLE`. Field errors point to stable JSON paths. Log request IDs but not secrets.

401 requires reauthentication; 403 requires manager/policy review; 409/422 requires review rather than blind retry; 429 honors `Retry-After`; 5xx retries the same document. A timeout has unknown outcome: query/retry using the original key, not “create again”.

Sale response and reconciliation

A success response returns `sale_id`, `client_sale_id`, `receipt_no`, `posted_at`, `business_date`, `total_minor`, `sync_cursor` and any `review_flags`. Return 201 for a new committed sale and 200 for an identical replay. A receipt is exposed only after commit. Canonical sales are immutable; refund creates another resource.

Input validation and limits

Strings have explicit max lengths; input names are text, not HTML. Query page size is 1–100. Sale lines are 1–100 with whole-unit quantity 1–999. Reject additional properties on money-sensitive command objects unless explicitly versioned. Reject duplicate line IDs and incompatible currency. API payload cap is a proposed 256 KiB per sale command; images upload separately with validated file type and dimensions.

Compatibility policy

Include `schema_version` in frozen offline payloads. Support the previous offline schema for the maximum permit window plus recovery grace; migration/deprecation decisions belong in an ADR. Never deploy a backend that rejects all queued documents from the previous client without a reconciliation path. Price revision tombstones remain readable for history even after a product is archived.

Examples

The canonical demo is four units across three lines, due SGD 14.00, tender SGD 20.00, change SGD 6.00. No tax or discount. Client-generated UUIDs in the fixture identify synthetic data only. A real client must generate its own stable IDs once and persist them before retries.

09 · Security, privacy and recovery

中文重点：这不是合规认证。先确保权限、历史数据和本机待同步销售不会被误处理。

Threat model

Assets: transaction history, stock ledger, cashier identity, local pending sales, database backups and server credentials. Threats: stolen/shared terminal, malicious browser input, XSS, CSRF, replay/duplicate commands, cross-store object access, client price tampering, server outage, local storage loss and accidental destructive maintenance.

V1 has one store per deployment, but identifiers remain store-scoped. A public portfolio demo is a separate security domain with synthetic data and no access to real shop/ERP credentials. Do not infer consent to copy Globe3 customer data into GitHub.

Authorization matrix

Capability	Cashier	Manager
Browse selling catalogue / perform own-shift sales	Yes	Yes
View product cost / receive or adjust stock	No	Yes
Override discount beyond cashier permit	No	Online approval only
Refund / export recovery / backup restore	No	Yes, online
Close own reconciled shift	Yes, within policy	Yes
Change currency/tax/device policy	No	Yes, with audit

Server checks role and scope on every operation, including direct API calls. Hiding a button is not authorization. Protect cost fields by response projection, not only CSS.

Session and device controls

Use HttpOnly secure cookies, rotate session identifiers on sign-in/privilege change, enforce inactivity/absolute lifetimes and revoke server sessions on logout. Protect mutation requests from CSRF and authenticate WebSocket/event channels if later introduced. [S09]

Passwords are stored with an established adaptive password hashing library; rate-limit login and require manager recovery procedures. Avoid shared universal PINs. Device enrollment and offline permissions are named and revocable. A local lock screen is a convenience boundary on a trusted device, not equivalent to OS-level encryption or remote revocation.

Browser data

Cache the app shell separately from authorized business data. Do not cache authentication responses in the service worker. Offline catalogue excludes costs/customer PII by default. Clear eligible cached data on explicit logout only after unresolved sales are synchronized or securely exported; otherwise lock access and require manager intervention rather than deleting evidence.

Use a restrictive CSP, escape imported text, validate image uploads and avoid remote script CDNs. Client-side encryption with a key stored alongside data does not solve a compromised-browser threat; document key management before claiming encrypted offline storage. Browser persistence is not a backup. [S04]

Payment boundary

V1 records cash and external payment acknowledgements; it does not handle cardholder account data. Do not build a card-number form or imply PCI compliance. A bank/payment-gateway integration requires a separate provider contract, webhook signature checks, event deduplication, settlement reconciliation and a revised threat model. The UI prototype never transfers money.

Audit and observability

Append actor/device/time/action/subject/reason/request ID for stock, product pricing, discount exceptions, refunds, shift close and security changes. Redact cookies, passwords, permits and payment references from logs where unnecessary. Log an offline exception's UUID and reason without logging its entire payload publicly.

Alert on unresolved outbox items, repeated idempotency conflicts, negative balances, backup failures, long-open shifts and high authentication failure rates. These are proposed operational alerts, not automations created by this delivery.

Backup and restore

Back up PostgreSQL with an established logical or physical backup process, encrypt at rest/in transit, separate credentials and store a copy off the application host. Include uploaded product media and migration/build metadata. Database backups do not contain transactions still present only on a disconnected browser.

Suggested project targets: daily backup and a verified restore rehearsal before any pilot. An RPO/RTO target is not achieved until measured. Restore into a separate database/environment, run schema and financial/stock reconciliation checks, verify identities and only then schedule a controlled cutover. Never overwrite a live database as a troubleshooting shortcut.

Public release guardrails

No real people, live receipts, credentials, IP addresses or internal infrastructure details in the public repo or screenshots. Secret-scan commits and built frontend assets. Choose a project license after review; this kit deliberately makes no MIT-license assertion for the eventual app. The exploratory image's generated license badge is not authoritative.

10 · Test strategy and acceptance gates

中文重点：测试计划不是测试结果。本次交付的实际检查写在 validation/REPORT.md。

Layers

Unit tests cover integer money, discount bounds, exclusive tax rounding, return allocation and status transitions. Integration tests run against disposable PostgreSQL and verify transactional invariants, store scoping, idempotency, locks and migration behavior. Browser E2E tests cover complete user journeys, IndexedDB transactions, offline/reconnect and responsive behavior. Physical devices/printers get a separate manual matrix.

Use deterministic fixtures and explicit fault injection; do not rely only on randomly toggling Wi-Fi. Add a test endpoint/proxy in test environments to drop the response after database commit, abort local writes, return 401/409/429/5xx and interrupt update activation. Such controls must never be enabled in a public production build.

Critical acceptance scenarios

ID	Scenario	Required result
T01	Normal cash sale: 14.00 due, 20.00 tender	One sale/payment; 6.00 change; exact stock deduction
T02	Double-click confirm	One local document and one canonical sale
T03	Server commit succeeds, response is lost	Same UUID retry returns original; no second movement
T04	Same UUID with changed total	409; original preserved; visible review state
T05	Cash sale while offline then reload	Locally committed sale/outbox/receipt survive normal reload
T06	Storage quota/transaction abort	No success screen, no partial rows, cart preserved
T07	Price changes while cart is open	Review before payment; completed permitted snapshot preserved
T08	Offline replay exceeds current stock	Sale recorded with negative-stock exception; no disappearance
T09	Two concurrent refunds of final unit	At most one succeeds; no over-refund/restock
T10	Partial return rounding, then remaining return	Cumulative refund equals original, never exceeds it
T11	Product edit/archive after sale	Historical receipt unchanged
T12	Cross-store IDs / cashier calls manager endpoint	Rejected; no leaked cost or record existence
T13	Shift close with pending/review sales	Close blocked; cash-count draft remains
T14	Update available during checkout/pending outbox	No forced refresh; safe update deferred

ID	Scenario	Required result
T15	Second writable tab / expired lease	Single writer; recovery uses same IDs
T16	Card/PayNow selected offline	Disabled with reason; no “verified” status
T17	No Background Sync API	Foreground/manual path fully functional
T18	Browser storage cleared / first launch offline	Honest unavailable state; no fabricated catalogue/sales
T19	Backup restored to isolated environment	Ledger/payment/receipt reconciliation passes
T20	Print fails or user reprints	Sale retained; no duplicate sale

The detailed CSV expands these scenarios with preconditions, steps, expected evidence and release stage. Traceability links requirements to tests and screens.

Responsive and accessibility matrix

Test 390×844, 430×932, 768×1024, 1024×768 and 1440×900; keyboard-only; 200% zoom; touch; long names; reduced motion; screen-reader landmarks/focus; empty/loading/error states. Automated accessibility tools do not replace manual payment-dialog and receipt reading checks. A WCAG 2.2 AA goal is not a compliance certificate. [S07]

Browser and hardware matrix

Desktop Chromium, Firefox and Safari; iOS Safari and installed home-screen mode; Android Chromium. Record exact versions and OS at execution. Service-worker/IndexedDB tests require HTTPS or trusted localhost. Safari/WebKit emulation does not prove actual iPhone standalone safe areas, OS suspension, installation or printer behavior. Mark those tests NOT RUN until verified on real devices.

Barcode baseline is a keyboard-wedge USB/Bluetooth scanner; test focus and scan suffix configuration. Camera scanner, silent ESC/POS printing, cash drawer and physical terminal integrations are outside the release claim.

Release gates

G0: scope/ADRs and current dependency versions documented. G1: catalogue/stock domain tests pass. G2: online cash flow plus immutable history and refund invariants pass. G3: offline atomicity/idempotent recovery/update tests pass. G4: responsive/accessibility/security/backup rehearsal pass. G5: reviewer demonstration and README evidence are reproducible from a clean clone.

Block pilot release for duplicate transactions, lost locally acknowledged data in tested normal-reload scenarios, unauthorized refunds, incorrect cash totals, cross-store access, stale-price silent repricing, or unreviewed destructive migrations. Lower-priority cosmetic issues may ship only with explicit notes.

Evidence format

Each test record contains test_id, git SHA, build ID, environment, browser/device, fixtures, expected/actual outcome, screenshot/log/report path and PASS/FAIL/NOT RUN. Never convert “planned” to “passed” because a document or screenshot exists. Performance reports state sample count and p95 calculation, not only a single fast run.

11 · Implementation roadmap and work breakdown

中文重点：按可验收的阶段开发，不给没有依据的“完成百分比”。

Delivery sequence

Stage	Outcome	Dependencies	Exit evidence
M0 · Contract	Agree scope, ADRs, target environment and fixtures	This kit	Reviewed assumptions; repo/branch identified
M1 · Foundation	App/API scaffolds, auth, shared money, migrations, CI	M0	Clean build; unit tests; empty DB migration
M2 · Catalogue	Products, price revisions, receive/adjust ledger	M1	Duplicate/stale update tests; stock reconciliation
M3 · Online sale	Cart, cash tender, posting, receipt and sales detail	M2	T01–T04, T11, T20 integration evidence
M4 · Returns/shift	Manager refunds, cash in/out, count/close	M3	Refund concurrency and cash reconciliation
M5 · Offline PWA	Permit, local atomic sale, outbox, replay, update UX	M3/M4	Fault-injected T05–T08, T14–T18
M6 · Portfolio	Overview, responsive polish, docs, clean demo deploy	All	Clean-clone run, actual browser/device matrix

Do not estimate calendar completion without inspecting the selected repo, implementation team and available environments. These stages are work order, not a promise of background development.

Suggested tickets

ENG-01 initialize workspace and CI; ENG-02 authentication/role projection; ENG-03 money and tax tests; ENG-04 schema/migration; ENG-05 products/pricing; ENG-06 stock movement service; ENG-07 cart and barcode; ENG-08 cash checkout and atomic posting; ENG-09 idempotent replay; ENG-10 receipts and browser print; ENG-11 refunds and cumulative allocation; ENG-12 shifts and cash movement; ENG-13 offline permit/readiness; ENG-14 IndexedDB commit and writer lease; ENG-15 push/pull sync and stock watermark; ENG-16 service-worker update/migration; ENG-17 dashboards and export; ENG-18 responsive/a11y; ENG-19 security and restore drill; ENG-20 demo evidence and README.

Each ticket states in-scope files/modules, invariants, tests, migration impact, screenshot states and rollback concerns. Agents should not independently implement competing money algorithms or duplicate a shared schema package.

Agent collaboration

Architect freezes contract deltas; Developer implements a narrow vertical slice; Reviewer checks business invariants and diff; QA executes independent replay/device scenarios. Roles may be handled by separate runs, but independence must not be invented. A self-authored test is not a third-party audit.

Inspect `git status`, branch and uncommitted work before editing. Create a feature branch/worktree without resetting the owner's changes. Keep commits reviewable. Do not auto-merge, deploy, rotate credentials or touch Globe3 without task-specific authorization. The implementation prompt in this kit is a handoff, not permission to execute it on an unknown machine.

Development shortcuts that are acceptable

Use synthetic fixture images and a seeded demo store; keep styling dependency-light; use an ordinary browser print dialog; defer sophisticated customer management. Build one store well before designing a hosted multi-tenant billing platform.

Shortcuts that are not acceptable

No “stock = stock – qty” outside a posting transaction; no localStorage-only sales store; no live payment claim for a manual selector; no SQL in UI components; no fixed shared admin password; no service-worker forced reload during payment; no pretending a browser-only demo has a durable backend.

Documentation maintenance

Update OpenAPI, SQL, business rules and fixtures in the same PR when behavior changes. ADRs record the reason and affected tests. Regenerate screenshots from the implemented build, then replace this kit's visual-study images only when they represent actual behavior. Maintain a visible “planned / implemented / verified” capability table in the eventual repo.

12 · Deployment, environments and operating runbook

中文重点：GitHub 放源码和展示；真正 POS 的 API 和数据库另外部署。

Environment separation

Local development uses disposable data. CI uses ephemeral databases. Public demo has synthetic data and limited privileges. Pilot/production has separate database, credentials, storage and backup policy. Never point a demo build at an ERP or real shop database.

Recommended eventual topology: HTTPS reverse proxy → static client + API container → private PostgreSQL; optional object storage for product images/backups. Start with a single host or managed service suitable for the workload, not a price estimate. Hosting selection and costs need current provider-specific research when requested.

GitHub Pages is a static hosting service: suitable for this documentation or a clearly browser-only portfolio demonstration, not for running the Node API or PostgreSQL. [S08] A real PWA has HTTPS/secure-origin and update considerations beyond uploading HTML. [S02]

Build and configuration

Pin runtime and package versions, commit lockfiles, build immutable frontend assets and label API/client/database migration versions. Use server-only secrets via a secret manager or environment injection. Public client configuration contains only API base and non-sensitive display values. No database passwords, manager secrets or private signing keys in frontend `VITE_*` variables.

Set HTML/service-worker caching to permit update checks; hashed immutable assets may be long-lived. Use a restrictive CSP and appropriate MIME/cache headers. `/health/live` checks process liveness without secrets; readiness verifies required dependencies without leaking internal connection details.

Release procedure

Review diff and migrations → build/test exact commit → back up and verify restore availability → deploy to staging → run sale/refund/offline-upgrade smoke suite → confirm no incompatible pending-client schemas → controlled release → observe logs/queue/negative-stock exceptions → document rollback build and migration compatibility.

A client update must wait for safe activation. Do not delete old caches or IndexedDB stores just to force users onto a new version. Database migrations use a separately authorized role; the runtime role should not own all schema objects.

Operational checks

Before opening: server reachable, terminal identified, catalogue current, local persistence self-test passed, clock/permit valid, opening float recorded. During the day: inspect pending count and last successful sync. At closing: drain queue, resolve reviews, count cash, record variance and close shift online.

If API is down, authorized cached cash selling can continue inside the permit boundary. If IndexedDB fails, stop checkout and use the owner's separate contingency procedure; do not promise that refreshing will recover unsaved sales. If the terminal is lost, restore only server-confirmed data plus any previously exported recovery package; disclose unsynced-data exposure.

Backup / restore rehearsal

Create an encrypted backup, checksum and build/migration manifest. Restore to an isolated target; verify rows, stock sums, sale/payment sums, refunds and receipt uniqueness. Exercise one post-restore sale in the isolated environment, then discard test changes. Measure duration and recoverable timestamp. Store evidence with the release.

Incident recovery

For duplicate suspicion, search by original UUID and request ID before retrying. For a sync conflict, preserve payload, receipt and local state; do not clear browser storage. For negative stock, inspect all referenced movements and physical count. For failed update, retain queued data and roll back to a compatible build. Every manual correction needs actor/reason/audit trail.

Scope of this delivery

No service, container, cloud bill, GitHub repository, domain, backup automation or scheduled monitoring has been created by generating this kit. The deployment model and commands to be implemented remain future work.

13 · Future ERP integration without ERP dependence

中文重点：POS 自己可以运作，ERP adapter 以后再接；不要直接写 Globe3 table。

Architectural seam

Define a separate adapter package, not ERP-specific fields throughout the cart. POS commits its own sale and inventory transaction first. A durable integration_outbox publishes `pos.sale.posted.v1`, `pos.refund.posted.v1` and selected stock events for asynchronous delivery. Checkout never waits for ERP availability.

No existing Globe3 schema, API permissions or posting rules were inspected for this kit. Any actual integration requires authorized discovery and review. Do not map arbitrary POS UUIDs into an ERP field based on a similar name.

Mapping model

Maintain `source_system + entity_type + external_id + pos_id` mappings for product, location, tax code, currency and payment method. Version mapping changes and preserve the values used by historical events. An unresolved mapping sends the integration event to review; the local sale stays posted.

Decide stock authority before enabling two-way sync. For a future ERP-managed catalogue, ERP may own product/master pricing while POS owns sale capture; physical stock movement ownership must be explicit. Do not let both systems independently decrement the same sale and then import each other's decrement.

Event envelope

`event_id`, `event_type`, `schema_version`, `store_id`, `occurred_at`, `sale_id` or `refund_id`, `currency`, immutable totals/lines, source references and correlation ID. Exclude cardholder data, secrets and unnecessary customer information. Sign/authenticate the transport appropriate to the receiving API.

Delivery is at least once; receiver deduplicates `event_id`. Record attempts, `next_retry_at`, error code and acknowledged external document ID. Separate transient availability errors from permanent mapping/validation errors. Replay uses the same event identity.

Reconciliation

Daily compare POS posted sales/refunds with ERP acknowledgements by event ID and totals. Show pending, failed, duplicate-suppressed and reconciled counts. Corrections use an explicit compensating event; they do not rewrite a posted POS sale. Include a dry-run preview before enabling write operations.

Expansion gates

Only add this adapter after the standalone cash/refund/offline flows pass. First prove product read mapping and sale export to a sandbox. Then verify currency/tax/payment mapping, inventory ownership and retry behavior. Production ERP writing requires separate authorization, secrets and rollback procedures.

14 · GitHub portfolio and demonstration

中文重点：展示真实能力，不把设计图当作已经完成的应用。

Repository presentation

Suggested repo slug is `pwa-pos-system`; it has not been created. Describe the eventual app as “A lightweight PWA point-of-sale and inventory ledger with a bounded offline cash workflow.” Keep the name and license pending until reviewed. Pin screenshots, architecture, setup, data reset procedure, test evidence, limitations and a demo link only after that link exists.

A credible README contains: problem and scope, capability status table, a short demo sequence, actual stack/versions, architecture and tradeoffs, local setup, migrations, tests, failure recovery, security boundary and roadmap. Do not imply experience with a technology merely because a generated scaffold imports it; explain the decisions and code you can defend.

Five-minute demonstration script

1. Explain single-store scope and show the product/stock distinction. Create a synthetic item and receive a quantity.
2. Sell the canonical 14.00 basket, take 20.00 cash, show 6.00 change, open receipt and inventory movement.
3. Show the pending-local label while disconnected, then restore connectivity and demonstrate replay of the same UUID.
4. Drop a server response after commit in the test harness and prove no duplicate stock deduction.
5. Return one item with manager permissions, show linked history and close a reconciled shift.

The included HTML prototype supports only the interaction study: product selection, search/category, cart quantities, a simulated cash payment and visual navigation. It is not this complete demo script and has no backend/IndexedDB/PWA guarantee.

Synthetic store

Sample store name: Everyday Store. Sample cashier: Alex. Currency SGD; tax disabled. Twelve generic products with original vector illustrations; no retailer data or endorsement. The default basket is 2 Cold Brew, 1 Oat Cookies and 1 Sparkling Water, totaling 14.00. Dashboard sample is a separate synthetic daily dataset: 36 orders, gross 486.00, refunds 18.00, net 468.00. Do not present these as collected shop performance.

Resume language: only after implementation evidence

Possible completed-project statement: “Built a responsive POS and stock ledger using TypeScript and PostgreSQL; implemented idempotent sales posting, role-controlled refunds and a tested IndexedDB outbox for bounded offline cash sales.” Add numerical performance or reliability results only after measuring them.

For the present delivery, accurate language is: “Prepared a PWA POS product/engineering specification and interactive UI study covering transaction integrity, offline recovery and inventory-ledger design.”

Reviewer questions to prepare

Why not localStorage? Why separate sale and sync state? How is duplicate replay prevented? What happens when money was accepted offline but stock changed? How does a partial refund handle a rounding remainder? Why is a local payment label not bank confirmation? Why are server and pending-local report totals separated? How is a new service worker prevented from erasing a current sale?

Public release hygiene

Only synthetic fixtures. No passwords, full bank references, real customer contacts or employer source code. Choose a license intentionally; keep third-party notices. A visual mockup can be published as a design concept, but clearly label its non-implemented states.

15 · Decision log, assumptions and risk register

中文重点：这里区分用户明确要求、这次建议和仍然待确认的事项。

Evidence and precedence

The user's confirmed requirements are: standalone PWA POS, GitHub portfolio value, small catalogue, sales/inventory essentials and strong design; then a full documentation ZIP with images, wireframes and design. All more specific choices below are proposed baselines, not claimed user approvals.

Authority order: accepted future change request → business rules and API contract → database service invariants → screen specification/design tokens → wireframes → rendered mockups → exploratory AI image. Resolve conflicts in an ADR and update linked tests; do not treat decorative text in a concept poster as an API contract.

Proposed decisions

ADR	Decision	Reason / consequence
ADR-01	Standalone single-store modular monolith	Small scope; fewer distributed failure modes
ADR-02	One active selling device in V1	Bounds offline stock and cash-session complexity
ADR-03	Cash-only offline; external methods recorded online	Avoid false electronic-payment verification
ADR-04	Local atomic outbox + server UUID/hash dedup	Recover retries without duplicated business effects
ADR-05	Stock ledger plus transactional balance projection	Auditable history with fast read performance
ADR-06	Integer minor-unit money; exclusive tax only, sample tax off	Deterministic sums; no implied tax compliance
ADR-07	Online manager refunds; immutable completed history	Traceable correction and bounded authorization
ADR-08	Foreground sync is baseline; Background Sync optional	Browser portability [S03]
ADR-09	Same-origin cookie auth; bounded signed offline permit	Clear server/browser trust boundary
ADR-10	Explicit safe-time service-worker activation	Prevent interruption or migration loss [K01]
ADR-11	React/TS/Vite + Fastify/PostgreSQL/Dexie proposal	Shared contracts and portfolio learning; version review needed
ADR-12	POS header fixed; long admin pages may hide/reveal	Documented exception to smart-header default [K01]

ADR	Decision	Reason / consequence
ADR-13	Light theme only in V1; preserve user zoom	Keep tested design manageable and accessible
ADR-14	No actual payment/ERP integration in this release	Honest delivery boundary

Open decisions before coding

Approve or replace working brand/repo, target hosting machine, actual package versions, required browsers/physical printer, cashier discount ceiling, offline-permit limits, tax/legal settings, data retention and project license. Defaults are already supplied so design work can proceed; implementation must record how these were resolved without inventing approvals.

Risk register

Risk	Impact	Mitigation	Release disposition
Browser storage eviction/device loss	Unuploaded sales lost	Readiness test, storage monitoring, fast sync, recovery export	Residual risk disclosed
Duplicate retry / multi-tab race	Double sale or stock deduction	Stable IDs, unique keys, transaction tests, writer lease	Blocking if untested
Price/stock change while offline	Variance after money accepted	Historical snapshots, permit scope, exception reconciliation	Controlled, not hidden
Manual payment mislabelled verified	False revenue assurance	Explicit labels; external reference; no gateway claim	Blocking UI defect
Refund race / rounding drift	Over-refund	Locked cumulative entitlement tests	Blocking financial defect
Update interrupts transaction	Corruption/lost work	Waiting-worker UX and migration/recovery tests	Blocking lifecycle defect
Public demo reaches live services	Privacy/financial exposure	Isolated data/credentials; no real integrations	Blocking security defect
Scope grows into ERP	Incomplete project	Stage gates and explicit deferred scope	Reject unreviewed additions
License/brand assumption	Misrepresentation	Working identity; no app license assigned	Resolve before public release

Honest completion model

A documentation bundle can be complete while application development has not begun. Report artefact generation/validation separately from production software readiness. No arbitrary overall project percentage is assigned here.

16 · Source register and provenance

Checked: 2 October 2026. Platform behavior may change; recheck supported versions at implementation and release. These sources support specific platform facts, not the entire proposed POS architecture. All writing and vector wireframes in this kit are newly authored; no Odoo code or product screenshots are copied.

ID	Primary reference	What it supports / limitation
S01	Odoo 19 POS and Workflow	Search-visible official descriptions of browser POS and daily workflow. Full page fetch timed out during this session; no claim to have audited Odoo internals, exact storage or payment code.
S02	MDN: Using Service Workers and web.dev: Update	Secure-origin lifecycle and cache/update concepts. Safe checkout deferral is this project's design decision.
S03	MDN: Background Synchronization API	Limited browser availability; optional background synchronization.
S04	MDN: Storage quotas and eviction and WebKit storage policy	Quotas, estimates, eviction and persistence caveats. No zero-loss promise.
S05	MDN: IndexedDB	Transactional browser storage for structured data.
S06	PostgreSQL: Explicit locking	Row-lock lifetime, conflicts and consistent lock ordering.
S07	W3C: WCAG 2.2 target size minimum	24px minimum criterion and exceptions; our 44px target is a stricter design choice.
S08	GitHub Docs: What is GitHub Pages?	Static hosting distinction.
S09	OWASP: Session Management	Secure session lifecycle and cookie considerations.
S10	Vite: Getting Started	Client build tooling and templates; no version pin chosen by this kit.
S11	Fastify documentation	Proposed API framework reference.
S12	Dexie: Getting Started	Library entry point; Dexie Cloud is not selected or required.
S13	OpenAPI 3.1.0	Machine-readable API contract format; intentionally not described as latest.

Private source: applied house standard

K01: KB-MCP skill `pwa:product-standard`, version 1.0.0, exact reference `270d896a-12b0-40b4-856f-f6274d872c95:3f932f5e-8bc9-40c6-86e9-e676d47e1c52`, read in this session. Applied requirements: opaque app chrome, safe areas, explicit user-controlled worker updates with loading feedback, versioned caches, defined offline contract, mobile/accessibility QA and no “manifest means done” claim. No unrelated private project details were included.

The initial bounded context assembly exceeded its response budget; the narrower memory read and exact skill fetch supplied the relevant PWA standard. No confirmed existing PWA POS repository or approved brand was retrieved.

Provenance labels

“Requirement” originates in the user's brief; “proposed decision” is new design guidance; “platform fact” refers to the sources above; “synthetic” means invented sample data; “validation result” refers only to executed bundle checks. Images are a visual study, not screenshots of a deployed application.

17 · Asset guide and handoff conventions

Visual assets

`design/screens/` contains deterministic browser-rendered high-fidelity PNG studies for desktop, tablet, mobile, payment, receipt and back-office states. `design/wireframes/` contains grayscale SVG sources and PNG exports. `design/diagrams/` contains architecture, entity relationship and transaction/sync diagrams with editable sources. `design/tokens.*` expresses the canonical color/spacing/type direction.

The generated exploratory poster in `design/concepts/` is included for provenance only. It is not a chosen brand, app license, verified architecture, legal tax setup or supported payment integration. Its generated text is not a specification. Use the deterministic screens and business contracts for implementation.

Local HTML study

`prototype/index.html` is a dependency-free, in-memory UI study, not the proposed React implementation. It supports navigation, search/category filtering, add/remove quantity, mobile cart, cash tender/change, simulated receipt and an offline-status simulation. Offline switch means visual simulation only. It does not use IndexedDB, a server, a payment provider, a real service worker or a production login. Data resets on refresh by design.

Open the extracted HTML directly in a browser. Screenshot mode is available with query parameters documented in the prototype source; images were generated from these same layouts to reduce handoff ambiguity. Visible demo disclosures must remain when the study is shown publicly.

Editing and import

SVG files can be edited in a vector editor that supports SVG; exact import behavior is editor-dependent and has not been certified for Figma. PNGs are previews. There is no `.fig` file and no bundled commercial font. Original vector product illustrations are illustrative placeholders, not product photography.

Markdown is the editable text source. The PDFs are reading editions. OpenAPI/SQL fixtures are reference contracts that need their own application and database tests. Read `validation/REPORT.md` before interpreting PASS labels.

Handoff checklist

The implementing Agent reads README, business rules, offline sync and security before coding. Confirm target repo and environment; inspect uncommitted work; preserve the owner's files. Treat generated source as reviewable scaffolding, not trusted production code. Keep source-of-truth documents, contracts, tests and screenshots aligned in each PR.